Trajectory Prediction of a Pegasus XL Sounding Rocket for Vertical Payload Deployment

C. Emmett, M. Forte, K. Graziose, K.C. Madden, B.C. Tiamson

The Cromwells, Newark, Delaware 19716, United States

I. Abstract

The team analyzed a Northrop Grumman Pegasus XL Launch Vehicle as a three-stage sounding rocket carrying a 3,000 kilogram payload through one-dimensional vertical launch accounting for gravity, drag, and varying atmospheric conditions. The first stage is an Orion 50S XL rocket; the second stage is an Orion 50XL rocket; and the third stage is an Orion 38 rocket. The specifications for each stage are given in the ATK Motor Catalog. All assumptions used are stated at the beginning of the methods section. The team used Python to calculate the maximum altitude, using Euler's method to iterate through velocity and altitude as functions of time. Code was developed for varying drag with changing velocity as well as varying gravity and atmospheric conditions with changing height. Code was also developed to calculate varying mach number with changing velocity and altitude. The maximum altitude for each stage was calculated separately, then summed for total maximum altitude. The final altitude calculated for the total trajectory of the rocket was 1,121 kilometers.

II. Nomenclature

A	=	vehicle cross-sectional area
c	=	effective exhaust velocity
$\bar{c}$	=	average launch effective exhaust velocity
C_D	=	drag coefficient
g	=	gravity
$\bar{g}$	=	average launch gravity
g_0	=	initial gravity
m_0	=	initial mass
R_0	=	initial radius from the center of Earth
t_p	=	time at power cut-off
t	=	time
v	=	vehicle velocity
h	=	altitude (height)
ρ	=	mass density of the atmosphere
ζ	=	propellant mass fraction

III. Introduction

The main objective of this study is to analyze the performance of a Northrop Grumman Pegasus XL Launch Vehicle used as a sounding rocket to launch a payload of 3000 kilograms in a vertical trajectory. Though the Pegasus is typically launched from a carrier aircraft, it has been reevaluated as a three-stage sounding rocket for this study. The team has developed code for a one-dimensional trajectory accounting for factors such as gravity, drag, and the effects of the changing atmosphere as the vehicle ascends, to calculate the maximum altitude of the launch vehicle. Several assumptions have been made to simplify these calculations, detailed in the Methods section. The trajectory has been designed for the following motors, which will define the three stages of launch: (1) Orion 50S XL; (2) Orion 50 XL; (3) Orion 38. The code uses data from the motor specifications for these motors according to the provided ATK Motor Catalog. This report outlines the methods, results analysis, and conclusions of this study, using fundamental rocket propulsion concepts and elementary flight mechanics for multi-stage vehicles to determine the maximum altitude achieved by the launch vehicle.

IV. Methods

Assumptions

To simplify mathematical calculations, some major assumptions were made in this analysis. The section below lists them all:

1. The first assumption is that thrust is a constant for each stage according to the Orbital ATK Motor Catalog, from a time average. In reality, thrust comes from a vacuum thrust vs time curve, and nozzle exit pressure would have to be determined.

2. The second assumption is instantaneous stage separation and subsequent stage ignition. Again, in reality, this would not be possible as there is a moment between separation, while the stage falls away, and ignition. This is a brief period of coasting and is done also to save a small amount of propellant. This analysis assumes they happen at the same time as that timing would be hard to estimate.

3. Figure 4-4 from the textbook referenced shows the coefficient of drag versus Mach number for a range from 0.2 to 5.4 for a German V-2 Rocket at vertical flight. This analysis uses an interpolation in accordance with that graph and assumes anything above the Mach number of 5.4 to have a coefficient of drag of 0.15, the maximum value from the text [1].

4. This analysis also assumes a perfectly vertical, sounding, trajectory. This means all motion accounted for is in the perfect vertical direction. Without this assumption, gravity and coefficient of drag would also be functions of the angle of attack, something far too complicated for the scope of this course.

5. Temperature and air density are given by a function from NASA. The function considers one relationship for altitudes below 11,000 meters, another between 11,000 and 25,000, and another for anything higher than that [2].

Dynamical Equation

The equation below governs the dynamics of the system.

$$dv/dt = a(v,h,t) = \frac{c(h)\cdot\zeta/t_p}{1-\zeta\cdot t/t_p} - g(h) - \frac{1/2\cdot C_D(v)\cdot\rho(h)\cdot v^2 A/m_o}{1-\zeta\cdot t/t_p}, \quad [1]$$

which follows the nomenclature as provided in Section II. c, g, and ρ are a function of the altitude, h, and C_D is a function of Mach number, though it is denoted as a function of velocity, v, above.

Numerical Method

The numerical method chosen to integrate the dynamical equation above is called Euler's method. It was chosen for its simplicity as it is a first-order method. Euler's method is defined by Equation [2] below

$$y_{n+1} = y_n + hy'_n, \quad [2]$$

where y is some function, y' is the first derivative of that function, h is the size of the timestep, and n denotes which timestep. This method can only solve first-order differential equations. The governing dynamical equation is a second-order differential equation, so it must be reduced into a system of two first-order differential equations. This system is shown below.

$$v_{n+1} = v_n + a_n \cdot \Delta t, \quad [3]$$
$$h_{n+1} = h_n + v_n \cdot \Delta t.$$

The variables $a,\ v,\ h,$ represent the acceleration, velocity, and altitude of the rocket at timestep n.

Important Functions

Three functions are required to be defined in order to find the acceleration, shown below. The first is a function to find the gravitational acceleration at a given height, based on the Universal Gravitational constant, the radius of the Earth, height above the surface, and mass of the Earth.

$$g(h) = \frac{G * M_{earth}}{(R_0+h)^2} \quad [4]$$

Second is a function for temperature given the height, which is based on the NASA relation described earlier. Next is the density at a given altitude as well, coming from that same source, which uses another function for pressure.

$$T(h) = 288.19 - 0.00649h \text{ (h < 11,000)} \quad [5]$$
$$P(h) = 101.29 * \left(\frac{T}{288.08}\right)^{5.256}$$

$$T(h) = 216.69 \text{ (11,000 < h < 25,000)}$$
$$P(h) = 22.65 * e^{1.73-.000157h} \quad [6]$$

$$\text{T(h)} = 141.94 + 0.00299\text{h (h > 25,000)}$$
$$P(h) = 2.488 * \left(\frac{T}{216.6}\right)^{-11.388} \quad [7]$$

$$\rho(h) = \frac{P}{0.2869T} \quad [8]$$

Finally, a function for the Mach number was needed, as it changes with both temperature and vehicle speed. This function assumed a constant specific heat ratio throughout the flight and is given below:

$$Mach(T,u) = \frac{u}{\sqrt{\gamma \frac{R}{MW_{air}} T}} \quad [9]$$

Implementation

The code written aims to solve the function given in the Dynamical Equation section, as that is the governing equation of motion for the system. The difficult portion was solving the variables that are not treated as constants, needing to use time steps in order to iterate. The cross-sectional area is constant at each stage, given by the diameter of the fuselage. Density, Cd, and gravity change as a function of height, and drag force is dependent on velocity. This means that the acceleration is a function of both height and velocity.

To implement the program described, first initialization of all constants across the stages from the given data. This includes burn times of each stage, total impulse, average thrust, and mass among others. The initial height and velocity were given to be 0, as the rocket is launching from sea level while stationary.

Next, the functions described previously were defined outside of the main loop. This included the gravity at a given height due to Equation [4]. Next, a function to facilitate the drag coefficient interpolation was built using the aforementioned graph of the German V2 rocket. A function to calculate temperature and density at a given height was defined next, followed by the function for Mach number given temperature and velocity.

The main body of the code is a for loop iterating over a timescale for the flight. The time scale is given by adding the burn time of all the stages, and then adding some extra time at the end (about 15 minutes) to allow for coasting, and then dividing by a step of 0.05 seconds. To begin the loop, the stage is first determined, and then from that the current thrust and mass flux values are set, as well as cross-sectional areas from stage diameter.

Once this is set, the previous velocity and height from the last time step are taken to begin calculations. Mass is found by taking the previous mass and subtracting mass flux times the time step. Next, gravity, temperature, Mach number, drag coefficient, and density are calculated for the given conditions in order to get a thrust force and drag force. These forces now allow for a total acceleration given the current mass.

As previously mentioned, Euler's method for numerical integration was used, and the new velocity is the old velocity added to acceleration times the time step, and height is the old height added to the previous velocity times the time step. This continues, checking if the time passes burn time for any of the stages, and updating mass, area, and thrust according to the new parameters.

Once the rocket runs out of fuel, or the burn time is all used up, the thrust and mass flux values are both set to 0, with the mass being permanently set to the payload mass. This means the only acceleration being imparted on the payload is gravity, allowing it to coast back down through the atmosphere.

Once this loop is finished, a graph of the height as a function of time is plotted. Markers are placed for each stage separation, as well as a marker for the max height of the vehicle. Another graph is made for mass to show mass throughout the flight.

V. Results and Analysis

The code records the maximum height and velocity after each stage separates from the rocket. After the first stage separation, the maximum height of the rocket was 55,151 meters with a velocity of 2,037 meters per second. After separation of the second stage, the rocket reached a peak of 231,473 meters at a velocity of 3,222 meters per second. Following the third stage's separation, the rocket reached a peak of 449,243 meters at a speed of 3,237 meters per second. The overall maximum height of the rocket was 1121.5 km. Figure 1 shows the evolution of the rocket's mass as it goes through its flight. Figure 2 shows the rocket's altitude throughout the entire flight, noting the locations for maximum height as well as stage separations. Based on the information provided by the ATK Motor Catalog, the results for the rocket spending about 200 seconds going through its stage separations makes sense. It also makes sense that the peak altitude of the rocket would come much later while it is coasting in the atmosphere experiencing low drag to slow it down before slowing down enough to reach the ground.

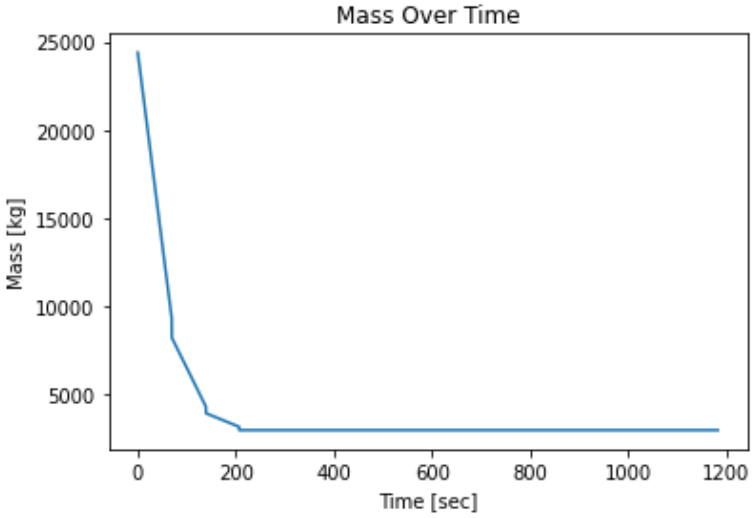

Figure 1 (Left): Mass vs Time of the Rocket through the entire flight.

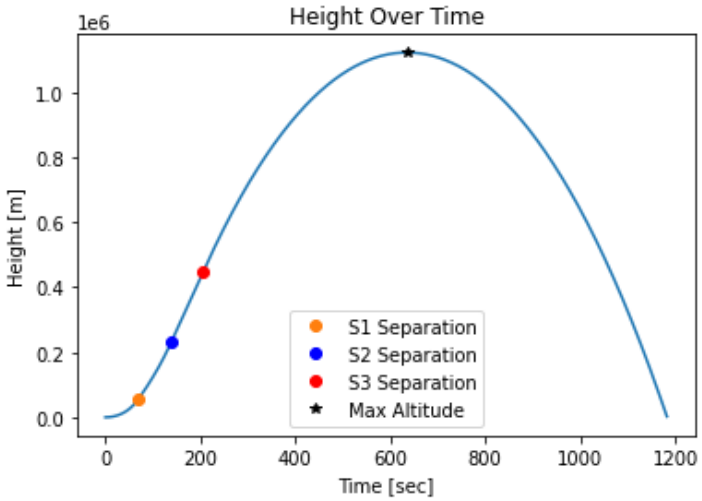

Figure 2 (Right): Altitude vs Time of the Rocket through the entire flight.

VI. Discussion

As stated before, our rocket reached a maximum altitude of 1121 kilometers. While most sounding rockets do not travel this high, this is not an uncommon distance. Some rockets have been known to travel as high as 1500 kilometers. Given the assumptions made in this analysis, it is safe to assume this is a maximum height, and the rocket would travel lower in the real world.

This analysis assumed a perfectly vertical flight path, something that isn't likely to happen in real life. Given the wind, the unpredictable nature of rocket engines, and other weather conditions, the rocket is unlikely to travel in a perfectly vertical path. This means that the rocket would not reach the height determined here.

This report assumed a constant thrust throughout the burn time of a stage. In reality, thrust changes with time as the mass flow rate changes while the engine burns and the exit velocity varies. This could either increase or decrease the height of the rocket given the times at which thrust is higher or lower and other factors such as drag and current height. While it is impossible to say how this assumption affected the outcome, it was a limitation overall in the accuracy.

The largest limitation of this simulation was the assumption for the drag coefficient. While the V2 Rocket was chosen as the comparison, these are missiles that would vary greatly from the rocket analyzed here. The drag coefficient is based on geometry among other factors such as the surface of the vessel. It was assumed that the drag coefficient changed with the Mach number, but the interpolation is likely not representative of the true drag going on.

Coasting between stages was another limiting factor, as stated earlier there is time while the stage moves away before the new one fires. This leaves a few seconds of coasting where the vehicle might begin to slow down, changing the overall flight path. This would likely be a minimal change but would affect the final numbers.

In conclusion, the maximum altitude of this craft was found using Euler’s method of iterating over the time of flight. The analysis used numerous assumptions but provides a good insight into how this would really fly. This sounding rocket would be good to take a payload out of Earth's atmosphere for a period of time, and the numbers for acceleration, velocity, and height are likely reasonable estimations.

Appendix A: Main Code

```
import numpy as np
import matplotlib.pyplot as plt

'''
This code is used to predict the trajectory of a 3-Stage Pegasus XL ground
launched vehicle in strict vertical flight, subjected to gravity and drag in the atmosphere.

First Stage: Orion 50S XL
Second Stage: Orion 50 XL
Third Stage: Orion 38

The following assumptions have been made to simplify the computation:
- Constant average thrust for each stage is used from the Orbital ATK Motor Catalog
- Instant stage separation and next stage ignition
- Drag coefficient is approximated versus Mach # for a range from 0.2 to 5.4. Anything above 5.4 is
estimated as 0.15
'''

################## Stage Parameters ####################
############## First Stage: Orion 50S XL ##############
s1_tp = 69.10 # stage 1 burn time (69.1s)
s1_impulse = 4.331E7 # stage 1 total impulse (N-sec)
s1_avgT = 6.263E5 # stage 1 burn time average thrust (N)
s1_massTotal = 16173 # stage 1 total mass (kg)
s1_massProp = 15023 # stage 1 propellant mass (kg)
s1_massBurn = 1092 # stage 1 burnout mass (kg)
s1_diam = 1.270 # stage 1 motor diameter (m)

s1_sep = s1_tp # time of stage 1 separation
s1_c = s1_avgT * s1_tp / s1_massProp # stage 1 exhaust velo
s1_mDot = s1_massProp / s1_tp # stage 1 mass flow rate

############### Second Stage: Orion 50 XL #############
s2_tp = 69.70 # stage 2 burn time (69.7s)
s2_impulse = 1.12E7 # stage 2 total impulse (N-sec)
s2_avgT = 1.606E5 # stage 2 burn time average thrust (N)
s2_massTotal = 4318 # stage 2 total mass (kg)
s2_massProp = 3923.6 # stage 2 propellant mass (kg)
s2_massBurn = 373.76 # stage 2 burnout mass (kg)
s2_diam = 1.270 # stage 2 motor diameter (m)

s2_sep = s2_tp + s1_sep # time of stage 2 separation
s2_c = s2_avgT * s2_tp / s2_massProp # stage 2 exhaust velo
s2_mDot = s2_massProp / s2_tp # stage 2 mass flow rate

############### Third Stage: Orion 38 ################
s3_tp = 67.70 # stage 3 burn time (67.7s)
s3_impulse = 2.184E6 # stage 3 total impulse (N-sec)
s3_avgT = 3.223E4 # stage 3 burn time average thrust (N)
s3_massTotal = 891.76 # stage 3 total mass (kg)
```

```
s3_massProp = 770.65 # stage 3 propellant mass (kg)
s3_massBurn = 110.22 # stage 3 burnout mass (kg)
s3_diam = 0.9652 # stage 3 motor diameter (m)

s3_sep = s3_tp + s2_sep # time of stage 3 separation
s3_c = s3_avgT * s3_tp / s3_massProp # stage 3 exhaust velo
s3_mDot = s3_massProp / s3_tp # stage 3 mass flow rate

########################## Payload ######################
pl_mass = 3000 # payload mass (kg)
#########################################################
'''
General Method
1. Iterate over timesteps
2. Calculate acceleration from force balance (forces change with time and height)
3. Use velocity and timestep to calculate height

Timeline
Stage 1 -> Stage 2 -> Stage 3 -> Burnout

du/dt = a = sX_avgT/total mass - g - 0.5*cd*rho*A*u**2/total mass

'''
'''
mass / (total mass - mass flow rate * time)
height / (velocity times time)
velocity / (acceleration times time)
acceleration / (forces)
density (height)
drag coeff (mach # -> velocity divided by acoustic speed (based off atmospheric temperature vs height))
gravity (height)
area (with stage)
'''
##### Thrust #####
'''
sX_avgT/sX_instM

s1_instM = (pl + s3 + s2 + (s1 - mdot*s1_t))
s2_instM = (pl + s3 + (s2 - mdot*s2_t))
s3_instM = (pl + (s3 - mdot*s3_t))
pl_instM = pl
'''
#### Drag ####
'''
drag = 0.5 * rho * CD * u**2 * A / sX_instM

s1_instM = (pl + s3 + s2 + (s1 - mdot*s1_t))
s2_instM = (pl + s3 + (s2 - mdot*s2_t))
s3_instM = (pl + (s3 - mdot*s3_t))
pl_instM = pl
'''
```

```
#### Gravity ####
'''
g = G*ME / ((RE + h)**2)
'''
def grav(h):
    G =  6.674E-11 # m^3/kg*s^2
    ME = 5.972E24 # kg
    RE = 6.371E6 # m
    g = G*ME / ((RE + h)**2)
    return g

#### Drag Coefficient ####
'''
Figure 4-4 from textbook for 0 deg angle of attack
Interpolate for in betweens
'''
def calcCD(mach):
    mach_nums = np.array(np.linspace(0.2,5.4,27))
    cd_coeffs =
np.array([0.15,0.15,0.15,0.18,0.30,0.41,0.36,0.30,0.26,0.25,0.24,0.23,0.22,0.21,0.21,0.20,0.20,0.19,0.18,0
.17,0.17,0.16,0.16,0.15,0.15,0.15,0.15])

    drag_coeffs = np.column_stack((mach_nums, cd_coeffs))
    if mach == 0:
        cd = 0
    elif mach >= 0.2 and mach <= 5.4:
        cd = np.interp(mach, drag_coeffs[:,0], drag_coeffs[:,1])
    else:
        cd = 0.15

    return cd

#### Density & Temperature ####
def atmos(h):
    # Returns air temp T [K] and density rho [kg/m^3]
    # Dependent on altitude (h) in meters
    # https://www.grc.nasa.gov/www/k-12/airplane/atmosmet.html
    if h > 25000:
        T = 141.94 + 0.00299*h
        press = 2.488 * (T/216.6)**(-11.388)
    elif h > 11000 and h <= 25000:
        T = 216.69
        press = 22.65 * np.exp(1.73-0.000157*h)
    else:
        T = 288.19 - 0.00649*h
        press = 101.29 * (T/288.08)**5.256
    rho = press / (0.2869*(T))
    return [T,rho]
#### Mach # ####
'''
M = u / as (acoustic speed)
```

```
as = np.sqrt(R*Mass*T)
'''
def calcMach(T,u):
    R = 8314 # Universal Gas Constant
    airM = 28.96 # Molecular weight of air
    Rspe = R / airM
    gamma = 1.402 # Wolfram: Assume constant specific heat ratio
    mach = np.abs(u) / np.sqrt(gamma*(Rspe)*T)
    return mach

##############################################
#
massTotal = pl_mass + s3_massTotal + s2_massTotal + s1_massTotal  # total mass
masses_t = [massTotal] # array to track mass changing over timesteps

h_t = [0] # height / (velocity times time)
u_t = [0] # velocity / (acceleration times time)
''' UNUSED THUS FAR
a_t = [0] # acceleration / (forces)
rho_t = [] # density (height)
cd_t = [] # drag coeff (mach # -> velocity divided by acoustic speed (based off atmospheric temperature
vs height))
'''
# gravity (height)
# area (with stage)
#
##############################################
dt = 0.05 # time increment (0.05s)
burnTime = s1_tp + s2_tp + s3_tp # aka s3_sep
burnout_time = 976 # time to fall back down range...just a guess for now
timesteps = int((burnTime + burnout_time)/dt) # Right now I think this will only bring us up to 3rd stage
burnout (adding 2 will get to stage 3 sep)
time = np.linspace(0,burnTime + burnout_time,timesteps+1)

for i in range(timesteps):
    calcTime = round(i*dt,2) # calculates runtime (not a function)
    if calcTime <= s1_sep:
        # stage 1
        sX_avgT = s1_avgT
        sX_mDot = s1_mDot
        A = 0.25*np.pi*(s1_diam**2) # area calculated based off larger motor diameter
        if calcTime == s1_sep: # Stage 1 Separation
            masses_t[-1] -= s1_massBurn
            print('Stage 1 Separation Complete')
            max_height_after_s1 = np.max(h_t[int(s1_tp / dt):])
            print('Stage 1 Maximum Height [m]:', max_height_after_s1)
            print('Stage 1 Maximum Velocity [m/s]:')
            print(np.max(u_t))
            print ('\n')
    elif calcTime > s1_sep and calcTime <= (s2_sep):
```

```
    # stage 2
    sX_avgT = s2_avgT
    sX_mDot = s2_mDot
    A = 0.25*np.pi*(s2_diam**2)
    if calcTime == (s2_sep): # Stage 2 Separation
        masses_t[-1] -= s2_massBurn
        print('Stage 2 Separation Complete')
        max_height_after_s2 = np.max(h_t[int(s2_sep / dt):])
        print('Stage 2 Maximum Height [m]:',max_height_after_s2)
        print('Stage 2 Maximum Velocity [m/s]:')
        print(np.max(u_t))
        print ('\n')
elif calcTime > (s2_sep) and calcTime <= (s3_sep):
    # stage 3
    sX_avgT = s3_avgT
    sX_mDot = s3_mDot
    A = 0.25*np.pi*(s3_diam**2)
    if calcTime == (s3_sep): # Stage 3 Separation
        masses_t[-1] -= s3_massBurn
        print('Stage 3 Separation Complete')
        max_height_after_s3 = np.max(h_t[int(s3_sep/dt):])
        print('Stage 3 Maximum Height [m]:',max_height_after_s3)
        print('Stage 3 Maximum Velocity [m/s]:')
        print(np.max(u_t))
        print ('\n')
else:
    # burnout (no more thrust)
    sX_avgT = 0 # payload does not produce thrust
    sX_mDot = 0 # mass no longer changes
    masses_t[-1] = pl_mass # final mass change that ensures mass at burnout is only payload

u = u_t[i]
m = masses_t[i] - sX_mDot*dt # mass change due to propellant
masses_t.append(m)
h = h_t[i]

g = grav(h)
T, rho = atmos(h)

mach = calcMach(T,u)
cd = calcCD(mach)

thrust_force = sX_avgT/m
drag_force = (0.5*cd*rho*A*(u*np.abs(u))/m)

a = thrust_force - g - drag_force
new_u = u + a*dt
new_h = h + u*dt
u_t = np.append(u_t,new_u)
h_t = np.append(h_t,new_h)
```

```
print("Maximum Height")
print(np.max(h_t))
max_height_time = time[np.argmax(h_t)]


plt.figure()
plt.plot(time, masses_t)
plt.title('Mass Over Time')
plt.xlabel('Time [sec]')
plt.ylabel('Mass [kg]')
plt.show()


plt.plot(time,h_t)
plt.plot(s1_sep,h_t[int(s1_sep/dt)],'o', label = 'S1 Separation')
plt.plot((s2_sep),h_t[int((s2_sep)/dt)],'bo', label = 'S2 Separation')
plt.plot((s3_sep),h_t[int((s3_sep)/dt)],'ro', label = 'S3 Separation')
plt.plot(np.argmax(h_t)*dt,max(h_t),'k*', label = 'Max Altitude')
plt.legend()
plt.show()
```

References

[1] G. Sutton and O. Biblarz, Rocket Propulsion Elements. Hoboken, New Jersey: John Wiley & Sons, Inc. 2017

[2] "Earth atmosphere model - metric units," NASA, https://www.grc.nasa.gov/www/k-12/airplane/atmosmet.html (accessed May 16, 2024).